

Sorting

Data Structure(CSL202)

Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)

Bubble sort





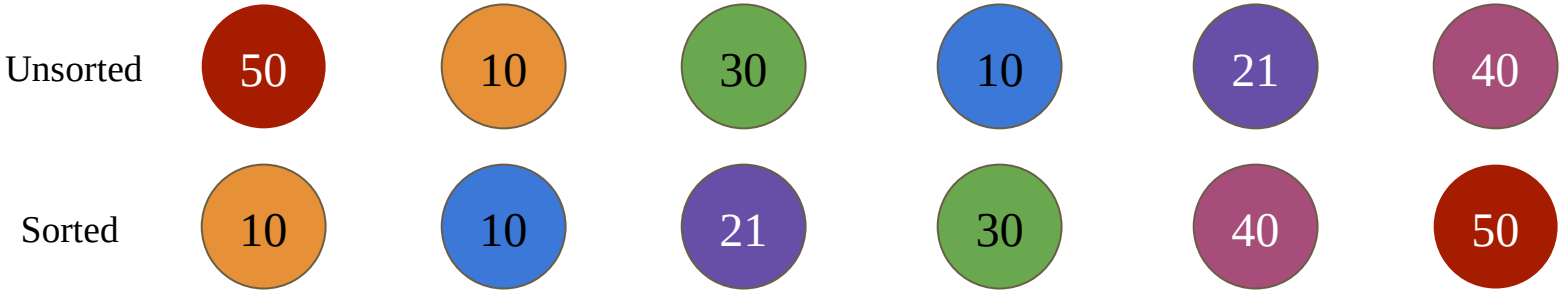
Bubble sort

- Repeatedly swap adjacent elements if they are in the wrong order(left element is greater).
- With each pass, the largest unsorted element "bubbles up" to its correct position at the end.
- **Time Complexity:**
  - **Worst/Average:** $O(n^2)$
  - **Best:** O(n) (if optimized with a "no swap" check)

```
void bubble_sort (int x[], int size)
{
        int t;
        for (i = 0; i < size; i++)
        {
                for (j = 0; j < size-i-1; j++)
                {
                        if (x[j] > x[j+1])
                        {
                        // swap a[j] and a[j+1]
                                t = a[j];
                                a[j] = a[j+1];
                                a[j+1] = t;
                        }
                }
}
```

Bubble sort

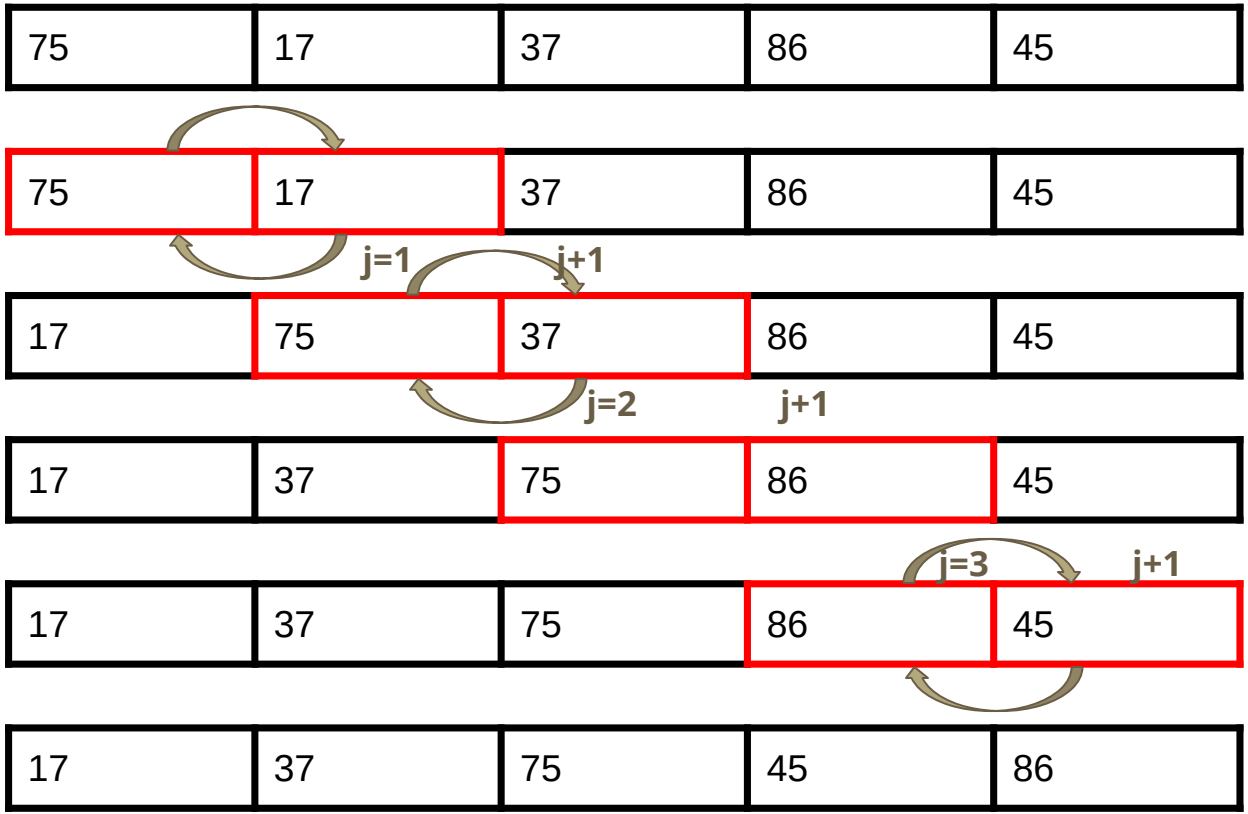
- **Stable Sort:** Keeps equal elements in original order.



- **In-place:**
  - Yes.
  - Uses no extra space (O(1)).

Example-

Given unsorted array:



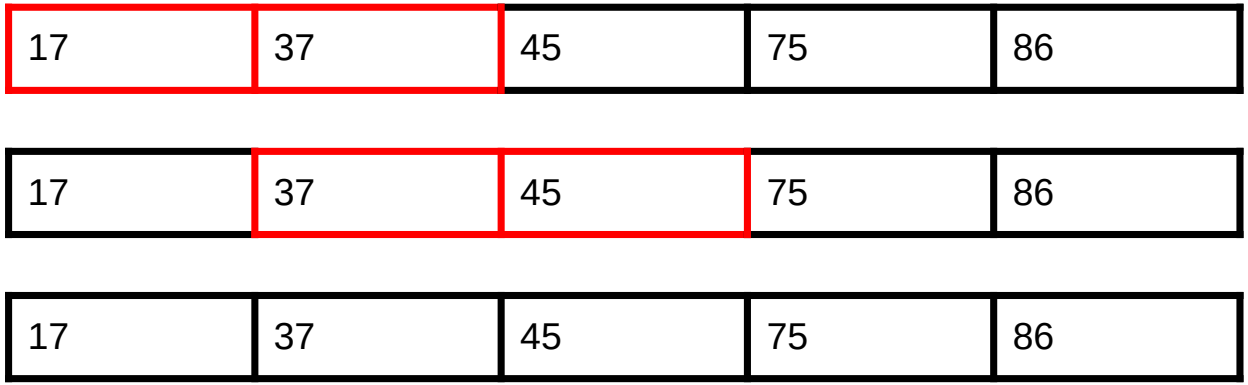
Example-

Pass 2:



Example-

Pass 3:



Pass 4:



Resultant sorted array.

Exercise

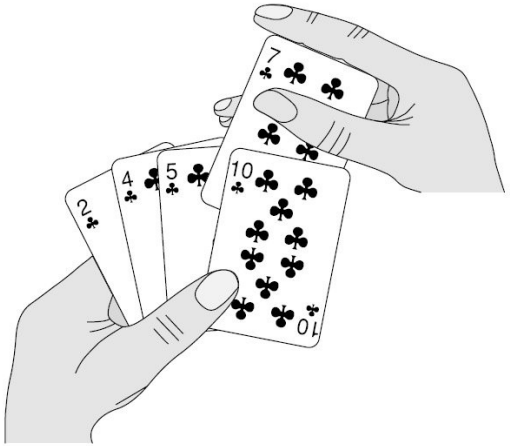
The number of swappings needed to sort the numbers 8, 22, 7, 9, 31, 5, 13 in ascending order, using bubble sort is?



Insertion sort

Insertion sort

- Much like how people sort playing cards in their hands.
- Picking up cards one by one and inserting each new card into its correct position among the already sorted cards in your hand.
- **Stable Sort:** Yes, doesn't change order of equal elements.
- **Online Sort:** Can sort while receiving input.
- **In-place:** Uses no extra space (O(1))
- **Time Complexity:**
  - **Worst-** Reverse sorted ($O(n^2)$).
  - **Average:** Random order($O(n^2)$).
  - **Best:** Array already sorted (O(n)).

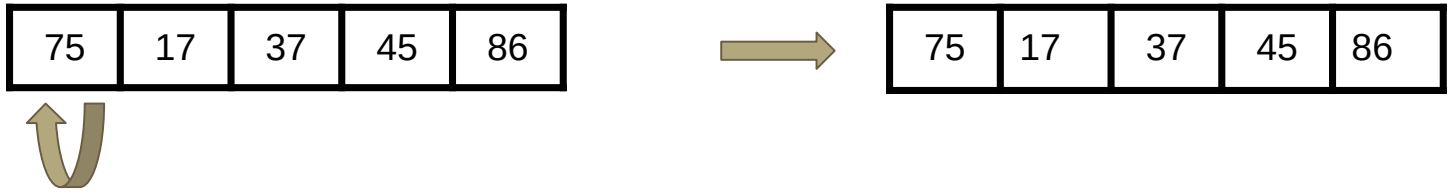


Insertion sort

```
void INSERTION(int A[], int N)
{
        for (int K = 1; K <= N; K++)                    //Start with the second element in the list.
        {
                        int TEMP = A[K];
                        int PTR = K;
                        // Shift elements to the right as long as TEMP is smaller
                        while (PTR>0  && TEMP < A[PTR-1])
                {
                        A[PTR] = A[PTR-1];        //Shift larger elements one position to the right.
                        PTR = PTR - 1;
                        }
                        A[PTR] = TEMP;                  // Insert TEMP in correct position
        }
}
```

Insertion sort

Pass 1:
PTR=K=0



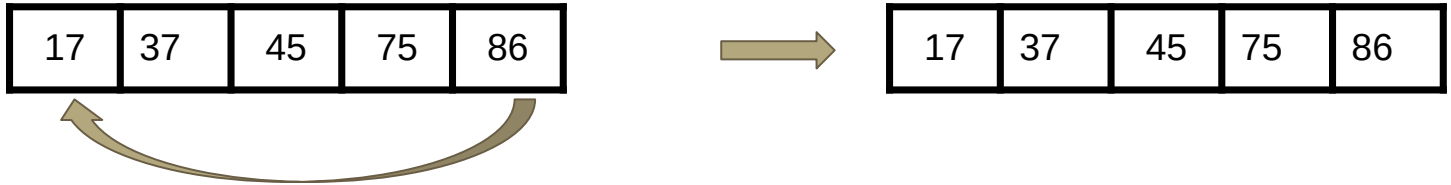
Pass 2:
PTR=K=1
Temp=17



Pass 4:
PTR=K=3
Temp=45



Pass 5:
PTR=K=4
Temp=86



Exercise

Consider the array A[]= {6,4,8,1,3} apply the insertion sort to sort the array . Consider the cost associated with each sort is 25 rupees , what is the total cost of the insertion sort when element 1 reaches the first position of the array ?

Selection Sort

- Simple comparison-based sorting algorithm.
- Divide the array into sorted and unsorted parts.
- In each pass:
  - Find the smallest element in the unsorted part.
  - Swap it with the first unsorted element.
  - Now, that element is considered sorted.
- Repeat until the whole array is sorted.
- **Stable Sort:** No.
- **In-place:** Yes, (O(1))
- **Time Complexity:**
  - **Worst-** sorted in descending order ($O(n^2)$).
  - **Average:** Random order($O(n^2)$).
  - **Best:** Already sorted in ascending order ($O(n^2)$).

Selection Sort

```
SELECTION(A, N):
1. For K = 1 to N-1:
   a. Call MIN(A, K, N, LOC)
   b. Swap A[K] and A[LOC]
2. End
```

```
Procedure MIN(A, K, N, LOC):
1. MIN := A[K]; LOC := K
2. For J = K+1 to N:
     If A[J] < MIN:
          MIN := A[J]
          LOC := J
3. Return LOC
```

Selection Sort

Pass	Location of min element in unsorted part	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]
1	LOC = 8	90	28	33	76	15	62	48	9
2	LOC = 5	9	28	33	76	15	62	48	90
3	LOC = 5	9	15	33	76	28	62	48	90
4	LOC = 5	9	15	28	76	33	62	48	90
5	LOC = 7	9	15	28	33	76	62	48	90
6	LOC = 6	9	15	28	33	48	62	76	90
7	LOC = 7	9	15	28	33	48	62	76	90
Sorted		9	15	28	33	48	62	76	90